

软件测试

三角形程序Triangle



输入 a、b、c, 范围 1–100, 输出等边、等腰、普通、无效。

如果只允许你测 5 次, 你选哪 5 组数据?

三角形程序的需求规格

- 三角形程序 Triangle 是软件测试教学中使用最广泛的例子。根据三边确定三角形类型（等边、等腰、普通、无效）
- 程序的输出是由三边 确定的三角形类型： 等腰、等边、普通三角形或无效三角形。有时这个问题会扩展到 包括直角三角形作为第五种类型。

为了 便于讨论，我们限定输入范围为 1 到 100 。Triangle的简要需求规格如下所示。

- ① a, b, c 为1到100的整数，否则输出“无效输入值”
- ② a, b, c 满足任意两边之和大于第三边，否则输出“无效三角形”
- ③ a, b, c 三边相等输出“等边三角形”
- ④ a, b, c 两边相等输出“等腰三角形”
- ⑤ 其他情况输出“普通三角形”

思考与讨论

需求规格的精确性

需求规格的完备性

隐含的需求规格

三角形程序Triangle的Python语言实现

- ① a , b , c为1到100的整数, 否则输出“无效输入值”
- ② a , b , c满足任意两边之和大于第三边, 否则输出“无效三角形”
- ③ a , b , c三边相等输出“等边三角形”
- ④ a , b , c两边相等输出“等腰三角形”
- ⑤ 其他情况输出“普通三角形”

```
def triangle(a, b, c):  
    if not (1 <= a <= 100 and 1 <= b <= 100 and 1 <= c <= 100):  
        print("无效输入值")  
    elif not (a + b > c and b + c > a and c + a > b):  
        print("无效三角形")  
    elif a == b == c:  
        print("等边三角形")  
    elif a == b or b == c or c == a:  
        print("等腰三角形")  
    else:  
        print("普通三角形")
```

思考与讨论:

1. Triangle程序的需求规格与程序代码对应关系?
2. Triangle程序的基本特征与潜在的测试思路?

三角形程序Triangle的Python语言实现

```
def triangle(a, b, c):  
    if not (1 <= a <= 100 and 1 <= b <= 100 and 1 <= c <= 100):  
        print("无效输入值")  
    elif not (a + b > c and b + c > a and c + a > b):  
        print("无效三角形")  
    elif a == b == c:  
        print("等边三角形")  
    elif a == b or b == c or c == a:  
        print("等腰三角形")  
    else:  
        print("普通三角形")
```

- Python 的代码简洁，代码逻辑相对容易理解，测试覆盖度相对容易实现。
- 由于 Python 是动态类型语言，变量的数据类型在运行时才确定，因此在测试时要特别注意不同类型数据的组合和边界情况，确保函数在各种可能的数据类型输入下都能正确运行，避免出现类型错误或意外的行为。
- Python 项目往往依赖于大量的第三方库，不同版本的库可能会导致程序行为不一致。因此，需要确保测试环境与生产环境一致，包括 Python 解释器的版本、依赖库的版本等，可以使用虚拟环境来隔离不同项目的依赖，避免相互干扰。

需求完全一样，测试是不是也应该完全一样？

三角形程序Triangle的C语言实现

```
#include <stdio.h>
void triangle (int a, int b, int c) {
    if (!(1 <= a && a <= 100 && 1 <= b && b <= 100 && 1 <= c && c <= 100)) {
        printf("无效输入值\n");
    } else if (!(a + b > c && b + c > a && c + a > b)) {
        printf("无效三角形\n");
    } else if (a == b && b == c) {
        printf("等边三角形\n");
    } else if (a == b || b == c || c == a) {
        printf("等腰三角形\n");
    } else {
        printf("普通三角形\n");
    }
}
```

- 由于 C 语言更接近底层，在设计测试用例时需要考虑更多的边界情况和内存管理问题。在应用中测试用例的设计要结合 C 语言的特性，如指针操作、结构体等。
- 不同的 C 编译器可能对 C 语言标准的支持存在差异，编译出来的程序在行为上可能会有所不同。在测试时，要使用多种编译器进行编译和测试，确保程序在不同编译器下都能正常运行。
- C 程序在不同的操作系统和硬件平台上可能会有不同的表现，例如不同平台的字节序、数据类型的长度等可能不同。测试时要考虑这些平台差异，确保程序在目标平台上能够正确运行。

三角形程序Triangle的Java语言实现

```
public class TriangleClassifier {
    public static void triangle(int a, int b, int c) {
        if (!(1 <= a && a <= 100 && 1 <= b && b <= 100 && 1 <= c && c <= 100)) {
            System.out.println("无效输入值");
        } else if (!(a + b > c && b + c > a && c + a > b)) {
            System.out.println("无效三角形");
        } else if (a == b && b == c) {
            System.out.println("等边三角形");
        } else if (a == b || b == c || c == a) {
            System.out.println("等腰三角形");
        } else {
            System.out.println("普通三角形");
        }
    }
}
```

- Java 是面向对象的语言，测试用例设计可以更多地关注类和对象的交互。Java 有严格的类型检查，在设计测试用例时可以利用这一特性，确保输入的参数类型正确。
- 确保测试环境和生产环境使用的 JDK 版本一致，不同版本的 JDK 可能会对程序的行为产生影响，特别是一些新特性或缺陷修复可能会导致测试结果的差异。
- Java 项目通常依赖于许多第三方库，要保证测试环境中使用的依赖库版本与生产环境一致。可以使用 Maven 或 Gradle 等构建工具来管理项目的依赖，确保依赖的一致性和完整性。

三角形程序Triangle的仓颉语言实现

```
public class Triangle {  
    public static func triangle(a: Int8, b: Int8, c: Int8) {  
        if (!((a >= 1 && a <= 100) && (b >= 1 && b <= 100) && (c >= 1 && c <= 100))) {  
            println("无效输入值")  
        }  
        else if (!((a + b) > c) && ((b + c) > a) && ((c + a) > b)) {  
            println("无效三角形")  
        }  
        else if (a == b && b == c) {  
            println("等边三角形")  
        }  
        else if (a == b || b == c || c == a) {  
            println("等腰三角形")  
        }  
        else {  
            println("普通三角形 ")  
        }  
    }  
}
```

- 类型安全检测：作为静态强类型语言，仓颉在编译时进行严格的类型检查，能在早期发现类型不匹配等错误，减少运行时错误的发生。
- 丰富的测试工具链：仓颉提供丰富的工具链，为开发者提供全方位的测试支持，可方便地进行各种类型的测试，提升测试效率和质量。
- 仓颉与 Java、Python 类似，都有相应的测试框架来支持单元测试、集成测试等。例如 Java 有 JUnit、TestNG 等，Python 有 unittest、pytest 等，仓颉也有自己的测试框架，提供断言方法、测试发现、测试运行器等功能，方便开发者编写和执行测试用例。

三角形程序Triangle的Python语言实现

```
def triangle(a, b, c):  
    if not (1 <= a <= 100 and 1 <= b <= 100 and 1 <= c <= 100):  
        print("无效输入值")  
    elif not (a + b > c and b + c > a and c + a > b):  
        print("无效三角形")  
    elif a == b :  
        if b == c:  
            print("等边三角形")  
        elif a == b or b == c or c == a:  
            print("等腰三角形")  
    else:  
        print("普通三角形")
```

```
def triangle(a, b, c):  
    if not (1 <= a <= 100 and 1 <= b <= 100 and 1 <= c <= 100):  
        print("无效输入值")  
    elif not (a + b > c and b + c > a and c + a > b):  
        print("无效三角形")  
    elif a == b :  
        if b == c:  
            print("等边三角形")  
        else  
            print("等腰三角形")  
    elif a == b or b == c or c == a:  
        print("等腰三角形")  
    else:  
        print("普通三角形")
```

思考与讨论：相同需求规格的不同程序代码实现带来的测试设计差异性？

请 DeepSeek 给 Triangle 写 5 个功能等价、代码结构完全不同的实现。

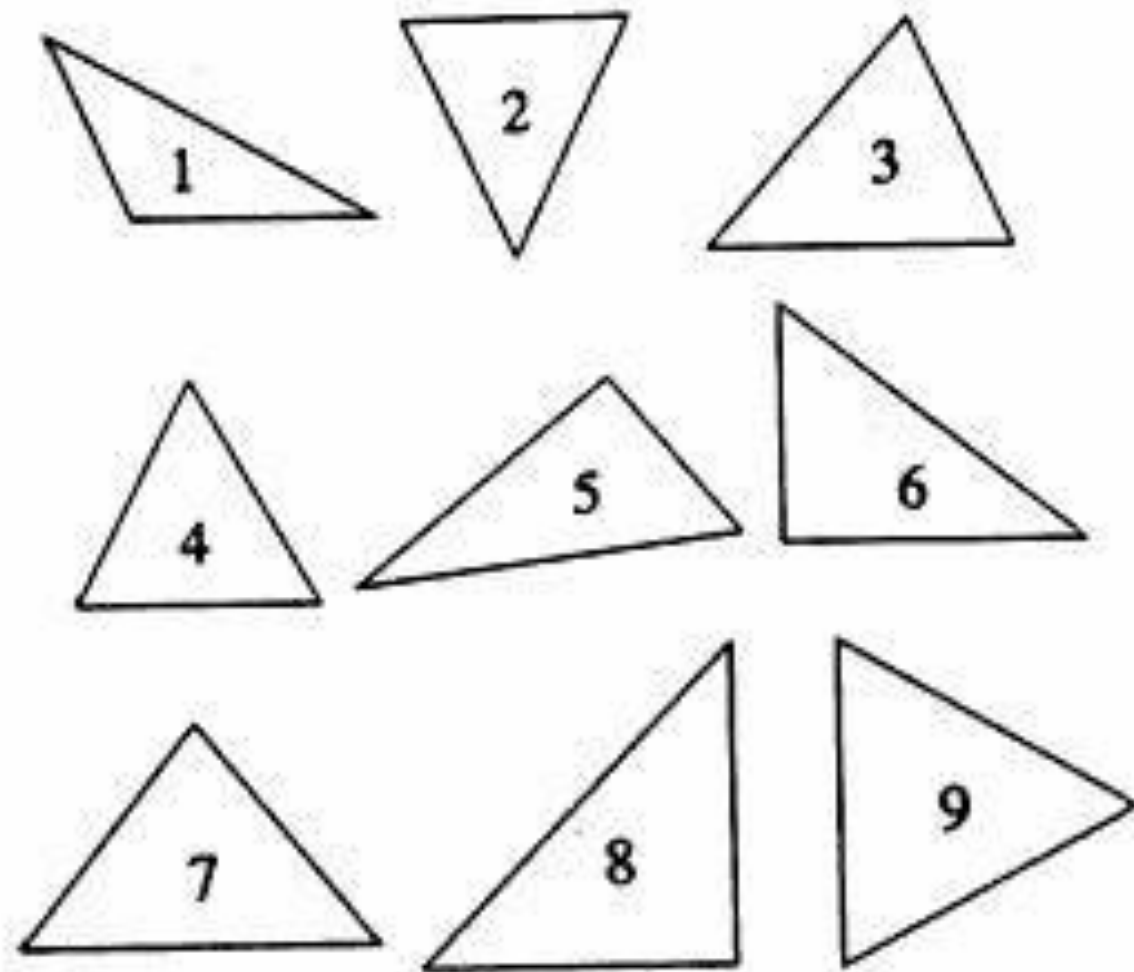
**如果以后软件大量由 LLM 生成，同一个需求可能瞬间得到
20 种实现，我们还能只靠开发者自己写几个测试吗？**

- ① a, b, c 为1到100的整数, 否则输出“无效输入值”
- ② a, b, c 满足任意两边之和大于第三边, 否则输出“无效三角形”
- ③ a, b, c 三边相等输出“等边三角形”
- ④ a, b, c 两边相等输出“等腰三角形”
- ⑤ 其他情况输出“普通三角形”

思考与讨论:

1. 训练一个分类器实现三角形分类, 记录不同分类器需要的样本量变化。
2. 训练一个神经网络实现三角形分类, 记录不同网络类型器需要的样本量变化。
3. 如何实现图片类的三角形分类算法?

回顾与讨论



测试设计与测试策略

- 程序特征对测试的影响
- 程序语言对测试的影响
- 确定性程序与机器学习

软件测试就是寻找这种“相信”的证据，也寻找“不应该相信”的反例。